BANZA: An Open Protocol for Financial Interoperability

Fidel R. Monteiro[1] and Jesus R. Monteiro[1]

[1]BANZAMI – Tecnologia e Serviços, Lda., Luanda, Angola
Official website: https://banza.network

Correspondence: Fidel R. Monteiro (contact@banza.network)

Abstract. Independent financial operators interoperate through bilateral integrations and shared infrastructures, but the specifications, the tests and the results are not always public, which makes technical validation hard to reproduce. This article presents BANZA, an open financial interoperability protocol grounded in public and versioned rules, in the separation between specification and implementation, in deterministic validation and in verifiable evidence. The protocol defines common contracts, schemas, profiles and mechanisms that allow implementations to be evaluated without conformance depending on the reference implementation, keeping the specification, the conformance evaluation and the participants' operational infrastructures separate. The normative surface of each version is explicitly indexed, signed or digested artifacts have a deterministic canonical representation, and the execution semantics — statuses, reason codes and idempotency — are public and accompanied by conformance vectors. Each result is bound to the inputs that produced it through canonical artifacts, cryptographic digests, reason codes and receipts, so that the technical basis of each evaluation can be inspected and reproduced by third parties. The protocol and the reference implementation remain in pre-production; independent third-party implementation, performance, scalability, adoption and behaviour in real operational environments have not yet been demonstrated experimentally.

1 Introduction

Financial operators rarely work in isolation. To transfer value or exchange messages, their implementations must agree on formats, identity, keys, error semantics and testing procedures. In bilateral integrations these elements are defined between each pair; when specifications, tests and results remain private, work is repeated and validation becomes difficult for third parties to reproduce.

Let n be the number of operators. Equations (1a) and (1b) represent, respectively, a mesh of bilateral integrations and the adoption of a common protocol.

$$R_{\text{bilateral}}(n) = \frac{n(n-1)}{2} \tag{1a}$$

$$I_{\text{common}}(n) = n \tag{1b}$$

Equation (1a) counts pairwise relations in a complete bilateral mesh — an illustrative case, not a universal model — and Equation (1b) counts implementations when each operator adopts the same specification: one more operator requires n new

The Portuguese-language edition is the canonical version of the BANZA Whitepaper. The English-language edition is an official translation. In the event of an unintended divergence, the canonical Portuguese edition prevails.

integrations in the first regime and one implementation in the second. Figure 1 distinguishes a common specification from a central infrastructure.

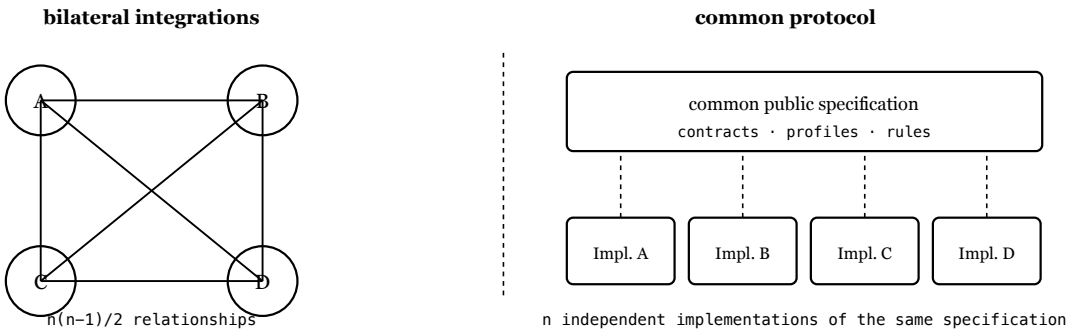

Figure 1. Complete bilateral mesh and common protocol. In the second regime the shared element is the public specification, not a central infrastructure.

A common protocol does not imply a common infrastructure. In the BANZA model each operator keeps its own implementation, infrastructure and operational relationships, provided it observes the specified behaviour and publishes the artifacts required by the version and profile it adopts. Interoperability can also be provided by shared infrastructures, which continue to perform routing, clearing or settlement; BANZA does not seek to replace them, but to add an open layer of specifications, profiles, versioned contracts, deterministic conformance and verifiable evidence, with which an implementation can coexist with different infrastructures, schemes or networks.

Standards such as ISO 20022 [1], ISO 8583 [2], the EMV specifications for QR payments [3] and ISO/IEC 9646 [4] address components of messaging, payments and conformance testing. Mojaloop [9] sits closer to BANZA: it combines an interoperability specification between providers — which admits either direct connection between them or connection mediated by a switch — with an open-source platform whose central services are used to run schemes. Both specify aspects of the same space and differ chiefly in how they organise the authority of the specification, the implementation, the conformance evidence and the operational scheme. BANZA complements them by defining an open protocol in which each implementation publishes artifacts at its canonical origin, is evaluated against a public version and profile, and produces results bound to the inputs observed — about implementations, not about entities.

This document is descriptive and non-normative: the requirements are defined by the versioned artifacts that the BANZA Normative Manifest identifies. What follows is the model and the architecture, the profiles, discovery and trust, validation and evidence, security and governance, the limitations and the current state, and the conclusions.

2 Model

BANZA distinguishes an operator — the responsible entity — from an implementation, the technical system being evaluated. One operator may publish several, and any result applies to a determinate implementation, version, profile, environment, scope, evidence set and validity period, never to an entity in the abstract.

Each implementation has a stable identifier, and the exact set of artifacts evaluated is fixed by a content digest: a different build is a distinct subject and requires its own evaluation.

We model an implementation as a six-element tuple. In Equation (2), o identifies the operator, i the implementation, v the protocol version, p the conformance profile, e the environment and u the canonical origin.

$$\mathcal{I} = (o, i, v, p, e, u) \tag{2}$$

The language, the database, the infrastructure provider and the origin of the code are not part of this tuple: conformance depends on the contracts and on observable behaviour.

The artifacts observed at the origin at an instant t form the set of Equation (3), where n is the number of artifacts.

$$A_t(\mathcal{I}) = \{\, a_1, a_2, \ldots, a_n \,\} \tag{3}$$

The normative surface of each version is not implicit: it is identified by the *BANZA Normative Manifest*, a versioned index declaring which artifacts define requirements. Determining what must be implemented is a lookup in published material, not an inference from the code.

Validation takes those artifacts and the specification S applicable to version v and profile p; engine version m produces the result R — verdict and reason code —, the evidence E and the receipts P.

$$V_m(A_t(\mathcal{I}), S_{v,p}) \to (R, E, P) \tag{4}$$

Scope is explicit: a result declares the profile, the version, the environment and the evidence consumed, holds only for that combination, and is not extrapolated to the entity that published the artifact. Reproducibility is central: given equivalent canonical inputs, the same specification, profile and engine version, independent executions produce semantically equivalent verdicts and reason codes. The motivation is that of reproducible research [8], but the equivalence is not an analogy — the protocol defines it normatively, requiring equality of the decisional statuses, of the bindings to the observed inputs and of the core reason codes. Non-deterministic metadata — timestamps, durations, execution identifiers and explanatory text — is excluded by definition, and byte equality is neither required nor used as the criterion.

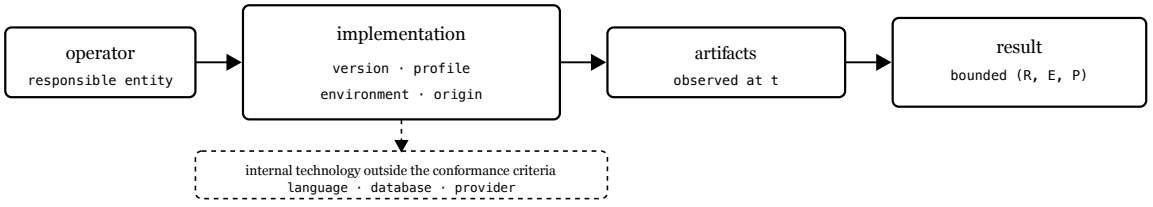

Figure 2. The result is bounded by the implementation and by the artifacts observed; internal technology is not part of the conformance criteria.

Content digests, computed with SHA-256 [5], identify and compare exactly the inputs observed: together with the availability of those inputs, they turn a validation claim into a technical basis third parties can re-evaluate. Comparing them requires equivalent representations to produce the same bytes. For JSON artifacts that are signed or digested, BANZA defines *BANZA*

Canonical JSON (BCJ/1), a restricted profile of the JSON Canonicalization Scheme [10]: a deterministic UTF-8 representation, members ordered by UTF-16 code units, integers within the safe domain $\pm(2^{53}-1)$, rejection of duplicate members rather than their resolution, and no verifier-side Unicode normalization. Duplicate rejection precedes semantic interpretation, because a parser that applies the first or the last occurrence has already lost the distinction the signature was meant to cover. Members not described by the schema are preserved and canonicalized like any other, and are therefore covered by the signature. The numeric domains of this version were audited and no field requires integers above the profile's domain.

3 Architecture

BANZA is organised in three institutional layers, separated by responsibility, infrastructure and keys. Layer 1 — Open protocol brings together specifications, contracts, profiles and common mechanisms for discovery, identity, trust, revocation and federation. Layer 2 — Conformance and Interoperability Certification evaluates implementations against public profiles and versions, with deterministic engines and verifiable evidence. Layer 3 — Independent operational schemes comprises infrastructures and networks that may adopt BANZA, subject to their own legal, commercial and regulatory rules.

Design decisions follow four Fundamental Principles — **BANZA $\mathbf{R^2S^2}$**: *robust*, correct and deterministic under independent implementation, adversarial input and boundary conditions; *resilient*, failures contained, safe operation preserved where possible and recovery deterministic without weakening guarantees; *secure*, critical properties enforced by construction, failing closed when they cannot be established; *simple*, the smallest mechanism sufficient for the required property. They are decision criteria, not a list of desirable qualities: resilience does not override security, nor promise continuous availability.

The principles decide; the eight *Protocol Structural Properties* developed in the Reference characterise what the protocol possesses — among them failing closed, associated with *secure* and *resilient* and not a fifth principle; the normative requirements define what an implementation satisfies; the evidence decides what may be claimed. Between them sit five Architectural Invariants, constraints the architecture may not violate: *open specification* — the applicable rules, contracts and profiles are public and versioned; *implementation independence* — no particular implementation constitutes the protocol, and the reference implementation realises it without defining it; *independent verification* — the published artifacts suffice for a third party to assess the applicable conditions without depending on a private decision; *operational independence* — no central infrastructure is required to carry messages or funds; and *separation of decisions* — conformance, technical certification, admission to a scheme, operational agreements and regulatory authorisation are distinct processes.

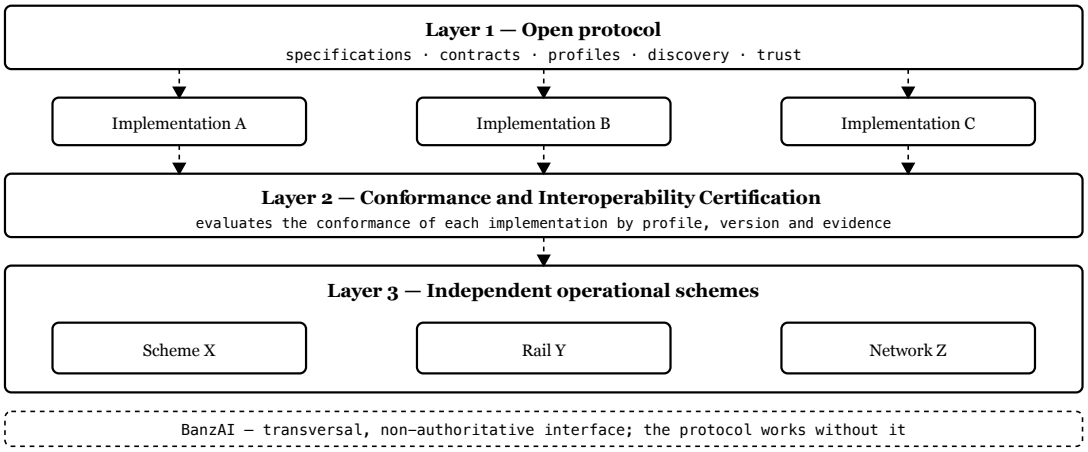

Figure 3. Layer 1 defines the specification, Layer 2 evaluates conformance through evidence, and Layer 3 remains operationally independent. The BanzAI is transversal, not a fourth layer.

Technical adoption consists of implementing the public specification and satisfying the applicable contracts: it requires neither the reference implementation's code, nor a connection to a central service, nor prior authorisation. The BanzAI is a human-facing interface for consultation and explanation, without taking part in the technical determination, and its unavailability blocks no protocol operation. The L0–L4 profiles of Section 4 are conformance levels, distinct from the three institutional layers.

4 Profiles

BANZA defines five cumulative levels, from L0 to L4. Adoption proceeds through the choice of a profile, the publication of artifacts at the canonical origin and the validation journey. A positive result is evidence that the technical conditions of the level are satisfied; it is not certification, authorisation or admission.

L0 — Protocol Sandbox validates safe configuration in a test environment, with a valid manifest and correct monetary representation. L1 — Core Payment Capability adds essential payment and traceability capabilities. L2 — Payment Initiation Capability adds initiation by request or dynamic QR code. L3 — Inter-Operator Interoperability adds requirements and evidence of interoperability between operators — routing, settlement and reconciliation, functions performed by the applicable operators or schemes — and constitutes the federation threshold.

The capabilities each profile requires are identified by a common normative vocabulary, and satisfaction is decided by exact comparison of the published identifiers, without implicit normalization; only normatively defined aliases are accepted. The public vectors have a scope determined by the profile: the association between case and level is declared, not inferred.

L4 — External Interoperability inherits L3 in full and adds no universal normative artifacts of its own: its requirements and evidence come from an external-interoperability profile identified by identifier and version, and the demonstration is confined to that profile. Satisfying L3 therefore does not satisfy L4 — with no profile selected the level remains unevaluated, not approved — and no concrete external profile is published.

Levels L0 to L2 can be assessed within a single operator. A certification profile is distinct from a level: it fixes a level and the capabilities, contracts and endpoints required, and is immutable once published.

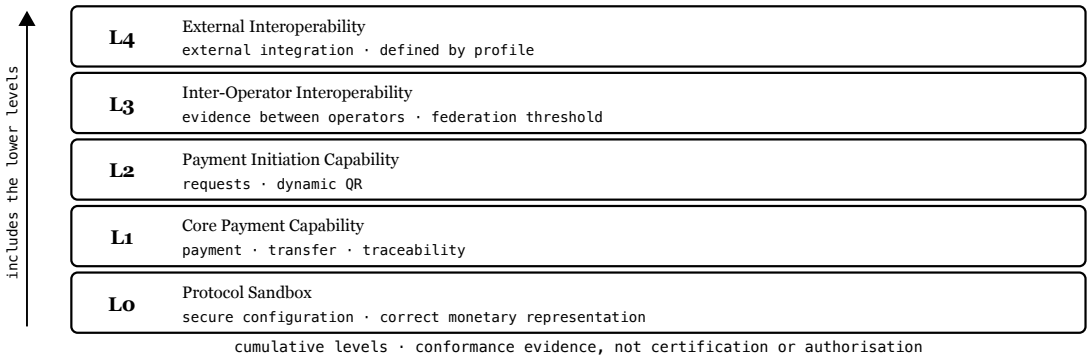

Figure 4. The profiles are cumulative. L3 adds evidence between operators and the technical federation threshold; L4 inherits it and depends on the selected external profile.

5 Discovery

Each implementation publishes its artifacts at a canonical origin controlled by the operator [7]. Discovery begins at a fixed path under that origin and returns what is needed to identify the implementation, the version, the profile, the endpoints, the signed metadata and the public keys. The Technical Registry may index them, without its presence being approval.

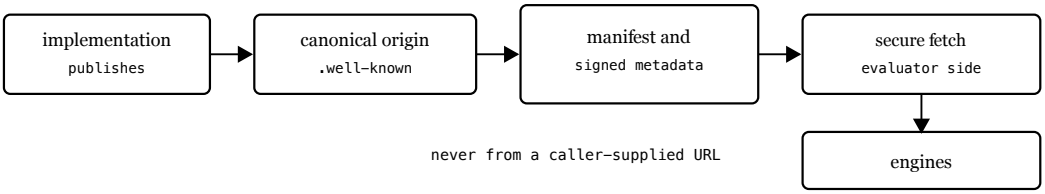

Figure 5. The implementation publishes at its canonical origin; the evaluator obtains the artifacts through a secure-fetch module. Discovery depends on no private decision.

Identity and integrity rest on published keys and signed metadata. The trust root is not a key: it is a set of three independent authorities, kept offline, where any authorised action requires two of them, distinct. The set advances as a lineage. The genesis set has sequence zero and no predecessor, and is accepted only when its digest equals one the verifier was explicitly configured with — trust on first use is refused. Every later set advances the sequence by one, names its predecessor by digest, and is authorised by signatures from two distinct authorities *of the predecessor set*, verified under the public keys as the predecessor records them: the candidate does not authorise itself. Continuity follows — if an authority is lost, compromised or obstructive, the remaining two authorise a successor that replaces it without its participation, since requiring it would make the path three-of-three and grant it a veto over its own replacement. Below the threshold, canonical continuity is blocked: there is no

emergency master key and no single-party path. The active set signs the Key Manifest with Ed25519 [6], which authorises delegated keys separated by domain; revocation and validity control reject revoked or expired material.

Trust material is versioned, and validity alone does not distinguish the current version from an earlier one still inside its period. The verifier therefore retains state about what it has already observed. Once material is accepted at an ordering point: a lower point is rejected; a higher, valid one becomes the accepted state; the same point with the same signing input is an idempotent observation; different content at the same point is equivocation, fails closed and leaves the state unchanged, so that the outcome does not depend on the order of retrieval. The property is local: it gives rollback protection and same-point conflict detection, and establishes neither a verifiable public history, nor global transparency, nor consistency across observers. Transport does not define trust — authority comes from the signed material and the lineage that authorises it, not from the origin that delivered it nor the order of arrival.

6 Validation

Validation follows a fixed journey of eight technical steps and one aggregation step, executed by deterministic engines. Each step ends in one of four terminal statuses — verified, pending, failed or blocked —, and no transient status is sealed into a receipt. Evaluation is fail-closed — absent, expired or inconsistent evidence never produces approval — and certification readiness requires every step of the profile verified, without constituting certification.

The separation between determination and explanation also exists within the result: the status determines the technical outcome, the reason code explains it. A conformant consumer derives its behaviour from the status, never changes a verdict because of a code and does not reject an artifact for failing to recognise one; extension codes are preserved in a namespace of their own and never interpreted. The explanatory vocabulary can thus be extended without changing decisions, and because statuses and codes are public, independent implementations reach semantically equivalent results over the same inputs.

The required behaviour is verifiable from published material: the contracts define inputs, outputs and observable statuses, and the public vectors fix cases with an expected outcome. For operations to which idempotency applies, the specification defines the key scope — receiving implementation, authenticated caller and operation —, what makes two requests the same request, conflicting reuse and the minimum retention.

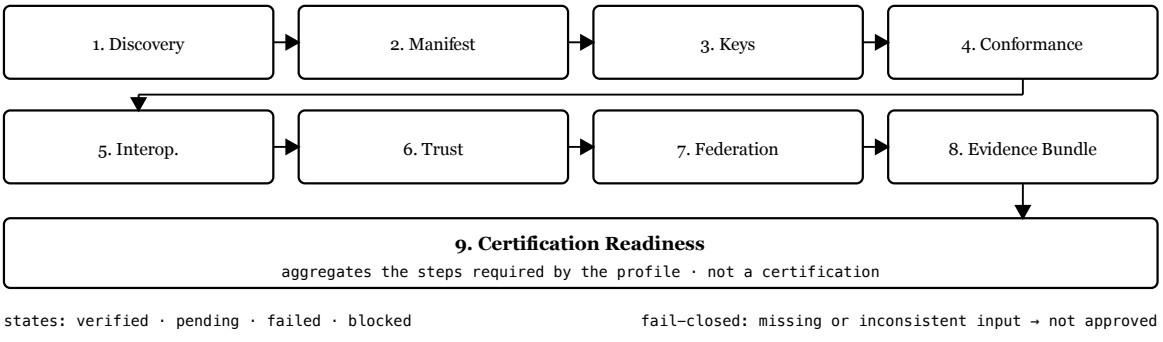

Figure 6. Eight technical steps and one aggregation step. Certification readiness aggregates the steps required by the applicable profile and does not constitute certification.

Figure 7 shows the directional evaluation of an implementation B by an evaluator A: B publishes its material, A obtains the artifacts from the canonical origin and evaluates locally. BANZA provides the verifiable technical basis; the operational use of the result remains under A's policy.

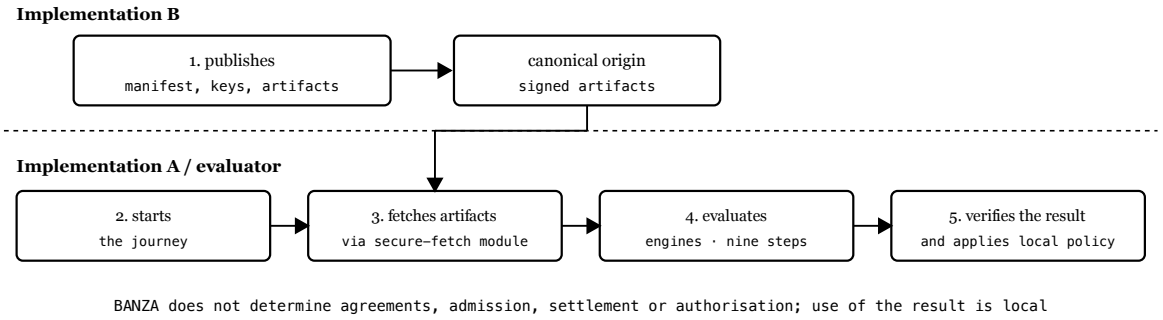

Figure 7. A→B evaluation: B publishes verifiable material, A evaluates locally and can reproduce the result. No central intermediary between A and B.

7 Evidence

Each validation produces results, receipts and verifiable evidence. The receipts record the steps and bind each result to the version, the profile, the artifacts consumed, their digests and the reason codes, allowing the technical basis of the evaluation to be reproduced.

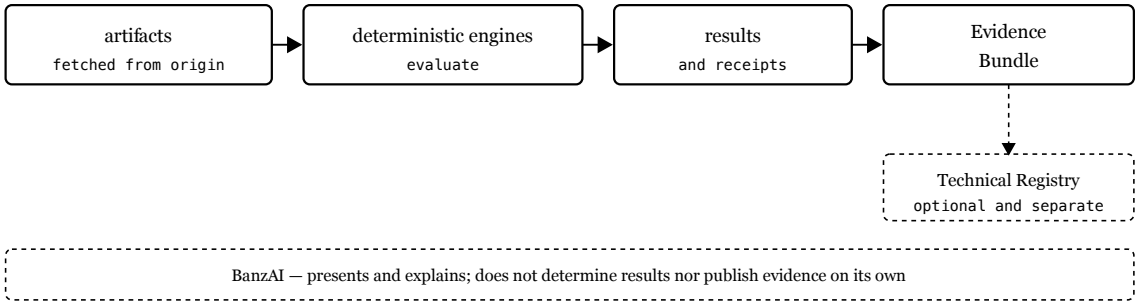

Figure 8. The engines evaluate artifacts and produce results and receipts; the Evidence Bundle gathers verifiable material. Publication in the Technical Registry is optional.

A receipt represents the state observed at the instant of evaluation; later changes require a new evaluation. The Evidence Bundle gathers the material that supports the result, without constituting certification or conferring operational status: in the Technical Registry, absence is not a prohibition and presence is not an authorisation. Independent verification requires the evidence to suffice for a third party to repeat the applicable steps without depending on any private decision.

The same criterion applies to the development of the protocol. A test is evidence only when it demonstrates the claimed property *for the intended reason*. Each critical property is therefore evaluated across independent layers, from normative authority to representation on the wire, to enforcement by the implementation, to behaviour under state, failure and attack,

and to derivability by a team without access to the engines: no layer validates itself, and a higher layer does not compensate for the failure of a lower one. Categories follow that do not imply one another: *specified*, *implemented*, *internally assured*, *independently implemented* and *operationally demonstrated*.

8 Security

The threat model pairs each threat with a mechanism: tampering is detected by signatures and digests; an invalid origin is excluded by resolution from the canonical origin; revoked or expired material is rejected; challenge replay is prevented by single-use challenges; SSRF and DNS rebinding are blocked by hardened remote fetching; absent or inconsistent evidence leads to fail-closed evaluation; and monotonic observation protects against regression and conflict at the same ordering point. None depends on implicit trust in the origin, and none substitutes for the others: the signature establishes authenticity, validity and revocation say whether the material is usable now, and the monotonic state prevents old material, still valid, from replacing more recent material already observed.

The secure-fetch module is the only component that reaches the operator's endpoints. The target is resolved from the canonical origin, never from an arbitrary URL supplied by the caller; the module accepts only HTTPS, blocks private, local and cloud-metadata addresses, connects only to previously validated addresses, follows no redirects, validates TLS and bounds responses — observable requirements, independent of internal technology.

Resilience is treated with the same rigour and subordinated to security. The failure classes considered range from the failure of a process, of persisted state or of an external dependency to the unavailability of a peer, network uncertainty and the duplicate delivery that follows from it, failure to publish trust material, and the loss of a Root authority. The intended behaviour is a progression — normal operation, safely degraded operation, localised fail-closed and deterministic recovery —, contained where the dependencies allow. But failing locally is not continuing unsafely: when trust, authorisation, integrity or correctness cannot be established, the correct response is to refuse. *Safety before availability* — no unavailability authorises accepting unsigned material, extending an expired validity or lowering a threshold, and a failure must never become a protocol violation.

9 Governance

BANZA evolves through public and versioned instruments: RFCs structure proposals, ADRs record architecture decisions. Versioning is explicit — incompatible changes require a new major version, compatible ones enter minor or patch versions — and each implementation declares the version and profile it adopts.

The openness of the protocol does not eliminate governance, but governance determines how the rules evolve without making the maintainer a technical intermediary between implementations: implementing a published version is distinct from having authority to change it.

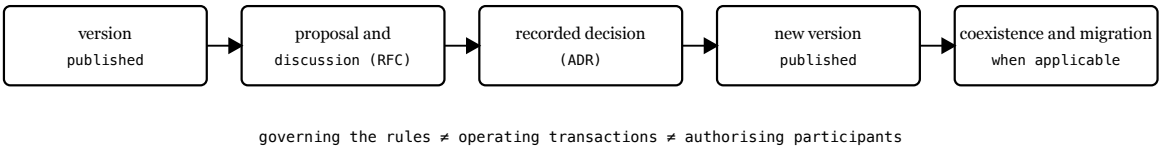

Figure 9. Protocol evolution separates proposal, recorded decision and versioned publication. Governing rules is not operating transactions nor authorising participants.

Incompatible changes are announced with a migration path, allowing temporary coexistence; deprecation is explicit and phased. The current version of the protocol is 1.0.0.

10 Limitations

BANZA's guarantees have clear boundaries. They are technical, not regulatory: a positive result demonstrates conformance with a public profile, without implying authorisation to operate, admission to a scheme or a commercial agreement.

Second, they apply to what is observed: reproducibility covers deterministic evaluations over the declared inputs, not the operator's internal controls nor behaviour outside the protocol, and a compromised origin or key is detectable only when it manifests in observable signals or in revocation material.

Third, the openness of the specification does not by itself demonstrate the ease of independent implementation: that property depends on the completeness of the contracts, the quality of the vectors and the ability of an external team to build a conformant implementation without recourse to the reference code. Likewise, the integration with external infrastructures envisaged by the architecture does not demonstrate effective access to those infrastructures, which may depend on interfaces, agreements, scheme rules or authorisation.

Fourth, some boundaries follow from the design of this version. Redundant distribution of already-signed artifacts would address availability and is not defined in 1.0.0: trust material is published at a single canonical point, and fail-closed evaluation preserves the correctness of the decision, but unavailable fresh material may prevent a new evaluation. The monotonic state of Section 5 is local, does not establish consistency across observers, and on a first observation staleness is not detectable. With no external profile published, L4 remains unevaluated. Finally, the evidence presented here is architectural: it demonstrates mechanisms in the reference implementation and does not measure performance, scale, availability in production or adoption.

11 State

BANZA remains in pre-production: the Technical Registry contains zero production operators and zero active technical certifications, payments with real money are disabled and the empty registry is the expected state. The reference implementation, Operator Zero, runs in an isolated test environment with demonstration currency, moves no real funds, remains uncertified and is evaluated by the same contracts and the same journey as any other implementation.

The normative surface is explicitly indexed, and the set required by each profile derives from that indexing. From it, clean-room material was derived for profile L0, without the reference implementation's code, and an internal verification — which

identified real questions, since corrected — concluded that it is closed upon itself, resolving all references without depending on the source repository, the decision records or the BanzAI. It is derived and internally validated material: as of this edition, the exact state to be fixed as the target of the external trial has not been selected or frozen, and no trial package has been published as a final artifact.

The completeness of a package and the capability for independent implementation are distinct properties: the first was verified internally, the second was not. Demonstrating it requires an implementation built by a genuinely independent team, from the public specifications and contracts, able to satisfy the applicable vectors and to take part in a directional evaluation with another implementation. That trial has not begun, no internal verification substitutes for it, and its success would still not amount to regulatory authorisation, admission to a scheme or use in production.

12 Conclusions

This work presented BANZA as an open financial interoperability protocol whose definition depends on no particular implementation. The main contributions are:

- evaluation of implementations against public versions and profiles rather than of entities in the abstract, with conformance depending on observable behaviour and not on the origin of the code;
- separation between protocol, certification and operational schemes, keeping technical conformance, admission, operational agreements and regulatory authorisation distinct, and distinguishing technical determination, operational decision and explanation assisted by the BanzAI;
- a public, versioned and explicitly indexed normative surface, with a deterministic canonical representation and public execution semantics — statuses, reason codes, idempotency — and vectors with a scope declared per profile;
- deterministic validation bound to the inputs through digests, receipts and verifiable evidence, with cumulative profiles and fail-closed evaluation;
- a trust root that is a set of authorities with a threshold and predecessor-authorised succession, rather than a single key, whose continuity never depends on a single party.

They rest on the five Architectural Invariants of Section 3 — open specification, implementation independence, independent verification, operational independence and separation of decisions —, which remain the criteria by which the architecture should be assessed, and articulate with the other planes without being confused with them: the $\mathbf{R^2S^2}$ principles guide design, the Structural Properties characterise the protocol, the Invariants bound the admissible architecture, the normative requirements define conformance, and layered evidence determines what may be claimed.

They define an architectural direction, not a maturity already attained: the boundaries of Section 10 remain to be lifted, and it is on them that future work bears — an independent implementation built by an external team, the interoperability between implementations so obtained, and the measurement of reproducibility, performance, scale and external profiles. In summary, BANZA makes the technical basis of interoperability public and verifiable without becoming a mandatory operational intermediary: different implementations adopt the same specification, demonstrate that they satisfy the applicable requirements and produce evidence that third parties can reproduce.

References

[1] International Organization for Standardization. ISO 20022 — Financial services — Universal financial industry message scheme. ISO. https://www.iso20022.org/

[2] International Organization for Standardization. ISO 8583:2023 — Financial-transaction-card-originated messages — Interchange message specifications. Edition 3. ISO, 2023. https://www.iso.org/standard/79451.html

[3] EMVCo, LLC. EMV QR Code Specification for Payment Systems (EMV QRCPS): Merchant-Presented Mode, Version 1.1. EMVCo, November 2020. https://www.emvco.com/

[4] International Organization for Standardization / International Electrotechnical Commission. ISO/IEC 9646-1:1994 — Information technology — Open Systems Interconnection — Conformance testing methodology and framework — Part 1: General concepts. ISO/IEC, 1994.

[5] National Institute of Standards and Technology (NIST). Secure Hash Standard (SHS). FIPS PUB 180-4. NIST, August 2015. DOI 10.6028/NIST.FIPS.180-4. https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf

[6] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IRTF, January 2017. DOI 10.17487/RFC8032. https://www.rfc-editor.org/rfc/rfc8032

[7] M. Nottingham. Well-Known Uniform Resource Identifiers (URIs). RFC 8615, IETF, May 2019. DOI 10.17487/RFC8615. https://www.rfc-editor.org/rfc/rfc8615

[8] R. Peng. Reproducible Research in Computational Science. Science, 334(6060):1226–1227, 2011. DOI 10.1126/science.1213847. https://doi.org/10.1126/science.1213847

[9] Mojaloop Foundation. Open API for FSP Interoperability (FSPIOP) Specification and Mojaloop open-source platform. https://docs.mojaloop.io/

[10] A. Rundgren, B. Jordan and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, June 2020. DOI 10.17487/RFC8785. https://www.rfc-editor.org/rfc/rfc8785